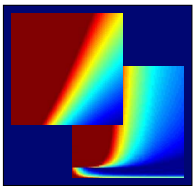
Om Sanan

CS/CNS/EE 156a Learning Systems
Caltech - Fall 2025
https://caltech.instructure.com/courses/8882
(Learning From Data campus version)



Homework # 7

Due Monday, November 17, 2025, at 2:00 PM PST

*Definitions and notation follow the lectures. All questions have multiple-choice answers (***[a], [b], [c], ...***). Collaboration is allowed but without discussing selected or excluded choices. Your solutions must be based on your own work. See the initial "***Course Description and Policies***" handout for important details about collaboration, open book, and chatGPT policies.*

Note about the homework

- Answer each question by deriving the answer (carries 6 points) then selecting from the multiple-choice answers (carries 4 points). You can select 1 or 2 of the multiple-choice answers for each question, but you will get 4 or 2 points, respectively, for a correct answer. See the initial "**Course Description and Policies**" handout for important details.
- The problems range from easy to difficult, and from practical to theoretical. Some problems require running a full experiment to arrive at the answer.
- The answer may not be obvious or numerically close to one of the choices, but one (and only one) choice will be correct if you follow the instructions precisely in each problem. You are encouraged to explore the problem further by experimenting with variations on these instructions, for the learning benefit.
- You are encouraged to take part in the Piazza discussion forum. Please make sure you don't discuss specific answers, or specific excluded answers, before the homework is due.

© 2012-2025 Yaser Abu-Mostafa. All rights reserved. No redistribution in any format. No translation or derivative products without written permission.

- **Validation**

In the following problems, use the data provided in the files `in.dta` and `out.dta` for Homework # 6. We are going to apply linear regression with a nonlinear transformation for classification (without regularization). The nonlinear transformation is given by ϕ_0 through ϕ_7 which transform (x_1, x_2) into

$$1 \qquad x_1 \qquad x_2 \qquad x_1^2 \qquad x_2^2 \qquad x_1x_2 \qquad |x_1 - x_2| \qquad |x_1 + x_2|$$

To illustrate how taking out points for validation affects the performance, we will consider the hypotheses trained on $\mathcal{D}_{\text{train}}$ (without restoring the full $\mathcal{D}$ for training after validation is done).

1. Split `in.dta` into training (first 25 examples) and validation (last 10 examples). Train on the 25 examples only, using the validation set of 10 examples to select between five models that apply linear regression to ϕ_0 through ϕ_k, with $k = 3, 4, 5, 6, 7$. For which model is the classification error on the validation set smallest?

[a] $k = 7$

[b] $k = 6$

[c] $k = 5$

[d] $k = 4$

[e] $k = 3$

Code below for 1-5

2. Evaluate the out-of-sample classification error using `out.dta` on the 5 models to see how well the validation set predicted the best of the 5 models. For which model is the out-of-sample classification error smallest?

[a] $k = 7$

[b] $k = 6$

[c] $k = 5$

[d] $k = 4$

[e] $k = 3$

3. Reverse the role of training and validation sets; now training with the last 10 examples and validating with the first 25 examples. For which model is the classification error on the validation set smallest?

[a] $k = 7$

[b] $k = 6$

[c] $k = 5$

[d] $k = 4$

[e] $k = 3$

4. Once again, evaluate the out-of-sample classification error using `out.dta` on the 5 models to see how well the validation set predicted the best of the 5 models. For which model is the out-of-sample classification error smallest?

[a] $k = 7$

[b] $k = 6$

[c] $k = 5$

[d] $k = 4$

[e] $k = 3$

5. What values are closest in Euclidean distance to the out-of-sample classification error obtained for the model chosen in Problems 1 and 3, respectively?

[a] 0.0, 0.1

[b] 0.1, 0.2

[c] 0.1, 0.3

[d] 0.2, 0.2

[e] 0.2, 0.3

- **Validation Bias**

6. Let e_1 and e_2 be independent random variables, distributed uniformly over the interval $[0, 1]$. Let $\mathsf{e} = \min(\mathsf{e}_1, \mathsf{e}_2)$. The expected values of $\mathsf{e}_1, \mathsf{e}_2, \mathsf{e}$ are closest to

[a] 0.5, 0.5, 0

[b] 0.5, 0.5, 0.1

[c] 0.5, 0.5, 0.25

[d] 0.5, 0.5, 0.4

[e] 0.5, 0.5, 0.5

E[e_1] = E[e_2] = 0.5 because uniform distribution
CDF of e is P(e ≤ t) = 1 - P(e_1 > t)P(e_2 > t) = 1 - (1-t)^2
PDF of e is f_e(t) = 2(1-t) for t in [0,1]
Expected value: E[e] = ∫_0^1 t*2(1-t) dt = ∫_0^1 2t-2t^2 dt = 1/3

- **Cross Validation**

7. You are given the data points (x, y): $(-1, 0), (\rho, 1), (1, 0)$, $\rho \geq 0$, and a choice between two models: constant $\{\, h_0(x) = b \,\}$ and linear $\{\, h_1(x) = ax + b \,\}$. For which value of ρ would the two models be tied using leave-one-out cross-validation with the squared error measure?

Constant:
Leave out (-1, 0): train on {(p,1), (1,0)} -> b=1/2 -> error = (0-1/2)^2 = 1/4
Leave out (p, 1): train on {(-1,0), (1,0)} -> b=0 -> error = (1-0)^2 = 1
Leave out (1, 0): train on {(-1,0), (p,1)} ->b=1/2 -> error = (0-1/2)^2 = 1/4
CV error = (1/4 + 1 + 1/4)/3=1/2

[a] $\sqrt{9+4\sqrt{6}}$

[b] $\sqrt{9-\sqrt{6}}$

[c] $\sqrt{\sqrt{3}+4}$

[d] $\sqrt{\sqrt{3}-1}$

[e] None of the above

Linear:
Leave out (-1, 0): train on {(p,1), (1,0)} -> ap+b=1, a+b=0, a=1/(p-1), b=-1/(p-1)
h_1(-1) = -2/(p-1) -> Error: 4/(p-1)^2
Leave out (p, 1): train on {(-1,0), (1,0)} -> -a+b=0, a+b=0, a=0, b=0
h_1(p) = 0 -> Error: 1
Leave out (1, 0): train on {(-1,0), (p,1)} -> -a+b=0, ap+b=1, a=1/(p+1), b=1/(p+1)
h_1(1) = 2/(p+1) -> Error: 4/(p+1)^2
CV error = [4/(p-1)^2 + 1 + 4/(p+1)^2]/3
[4/(p-1)^2 + 1 + 4/(p+1)^2]/3 = 1/2
p = ±sqrt(9+4*sqrt(6)) using CAS

- **PLA vs. SVM**

Notice: Quadratic Programming packages sometimes need tweaking and have numerical issues, and this is characteristic of packages you will use in practical ML situations. Your understanding of support vectors will help you get to the correct answers.

In the following problems, we compare PLA to SVM with hard margin[1] on linearly separable data sets. For each run, you will create your own target function f and data set $\mathcal{D}$. Take $d = 2$ and choose a random line in the plane as your target function f (do this by taking two random, uniformly distributed points on $[-1, 1] \times [-1, 1]$ and taking the line passing through them), where one side of the line maps to $+1$ and the other maps to -1. Choose the inputs $\mathbf{x}_n$ of the data set as random points in $\mathcal{X} = [-1, 1] \times [-1, 1]$, and evaluate the target function on each $\mathbf{x}_n$ to get the corresponding output y_n. If all data points are on one side of the line, discard the run and start a new run.

Start PLA with the all-zero vector and pick the misclassified point for each PLA iteration at random. Run PLA to find the final hypothesis g_{PLA} and measure the disagreement between f and g_{PLA} as $\mathbb{P}[f(\mathbf{x}) \neq g_{\text{PLA}}(\mathbf{x})]$ (you can either calculate this exactly, or approximate it by generating a sufficiently large, separate set of points to evaluate it). Now, run SVM on the same data to find the final hypothesis g_{SVM} by solving

$$\begin{aligned} \min_{\mathbf{w},b} \quad & \frac{1}{2}\mathbf{w}^{\mathrm{T}}\mathbf{w} \\ \text{s.t.} \quad & y_n\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b\right) \geq 1 \end{aligned}$$

using quadratic programming on the primal[2] or the dual problem, or using an SVM package. Measure the disagreement between f and g_{SVM} as $\mathbb{P}[f(\mathbf{x}) \neq g_{\text{SVM}}(\mathbf{x})]$, and count the number of support vectors you get in each run.

8. For $N = 10$, repeat the above experiment for 1000 runs. How often is g_{SVM} better than g_{PLA} in approximating f? The percentage of time is closest to:

[1] For hard margin in SVM packages, set $C \to \infty$ and choose 'linear' kernel.

[2] Primal problem is the original formulation in slide 11 of Lecture 14.

[a] 20%

[b] 40%

[c] 60%

See 8-10 code below.

[d] 80%

[e] 100%

9. For $N = 100$, repeat the above experiment for 1000 runs. How often is g_{SVM} better than g_{PLA} in approximating f? The percentage of time is closest to:

[a] 5%

[b] 25%

[c] 45%

[d] 65%

[e] 85%

10. For the case $N = 100$, which of the following is the closest to the average number of support vectors of g_{SVM} (averaged over the 1000 runs)?

[a] 2

[b] 3

[c] 5

[d] 10

[e] 20

Problems 1-5

```
import numpy as np

def load_data(filename):
    data = np.loadtxt(filename)
    X_raw = data[:, :2]
    y = data[:, 2]
    return X_raw, y

def transform(X_raw):
    """
    apply nonlinear transformation:
    Φ(x_1, x_2) = (1, x_1, x_2, x_1^2, x_2^2, x_1*x_2, |x_1 - x_2|, |
x_1 + x_2|)
    """
    x1 = X_raw[:, 0]
    x2 = X_raw[:, 1]

    # get N
    N = X_raw.shape[0]

    # create transformed feature matrix
    Z = np.c_[
        np.ones(N),
        x1,
        x2,
        x1**2,
        x2**2,
        x1 * x2,
        np.abs(x1 - x2),
        np.abs(x1 + x2)
    ]

    return Z

def calculate_classification_error(Z, y, w):
    """
    calculates classification error
    """
    # prediction scores
    y_scores = Z @ w

    # classify based on the score sign
    # g(x) = sign(w^T * z)
    y_pred = np.sign(y_scores)

    # edge case -> sign(0) = 0; assign to -1.
```

```python
    y_pred[y_pred == 0] = -1

    # fraction of misclassified points
    misclassified_count = np.sum(y_pred != y)
    error_rate = misclassified_count / len(y)

    return error_rate

def solve_linear_regression(Z_train, y_train):
    """
    solve standard linear regression using pseudo inverse
    """
    w = np.linalg.pinv(Z_train) @ y_train
    return w


if __name__ == "__main__":
    X_in_raw, y_in = load_data('./in.dta.txt')
    X_test_raw, y_test = load_data('./out.dta.txt')

    # split in.dta into training (first 25) and validation (last 10)
    X_train_raw = X_in_raw[:25]
    y_train = y_in[:25]
    X_val_raw = X_in_raw[25:]
    y_val = y_in[25:]

    print(f"Training set size: {len(y_train)}")
    print(f"Validation set size: {len(y_val)}")
    print(f"Test set size: {len(y_test)}")

    # Apply nonlinear transformation to all datasets; all 8 features
    Z_train_full = transform(X_train_raw)
    Z_val_full = transform(X_val_raw)
    Z_test_full = transform(X_test_raw)

    # P1
    print("\nProblem 1")

    k_values = [3, 4, 5, 6, 7]
    validation_errors = {}
    trained_models = {}

    for k in k_values:
        # use only first k+1 features
        Z_train = Z_train_full[:, :(k+1)]
        Z_val = Z_val_full[:, :(k+1)]

        # train regression on training set
        w = solve_linear_regression(Z_train, y_train)
```

```
        # validation error
        e_val = calculate_classification_error(Z_val, y_val, w)
        validation_errors[k] = e_val
        trained_models[k] = w

        print(f"k = {k}: E_val = {e_val:.6f} ({int(e_val *
len(y_val))}/{len(y_val)} errors)")

    best_k_validation = min(validation_errors,
key=validation_errors.get)
    print(f"\nBest model (smallest val error): k =
{best_k_validation}")
    print(f"Validation error:
{validation_errors[best_k_validation]:.6f}")

    # P2
    print("\nProblem 2")

    test_errors = {}

    for k in k_values:
        # Use only the first k+1 features for test set
        Z_test = Z_test_full[:, :(k+1)]

        # Use the model trained in P1
        w = trained_models[k]

        # Calculate eout
        e_out = calculate_classification_error(Z_test, y_test, w)
        test_errors[k] = e_out

        print(f"k = {k}: E_out = {e_out:.6f} ({int(e_out *
len(y_test))}/{len(y_test)} errors)")

    # find best model
    best_k_test = min(test_errors, key=test_errors.get)
    print(f"\nBest model (smallest E_out): k = {best_k_test}")
    print(f"Out-of-sample error: {test_errors[best_k_test]:.6f}")

    # P3
    print("\nProblem 3")

    # reverse role: train on last 10, validate on first 25
    X_train_raw_rev = X_in_raw[25:]
    y_train_rev = y_in[25:]
    X_val_raw_rev = X_in_raw[:25]
    y_val_rev = y_in[:25]

    print(f"Reversed - Training set size: {len(y_train_rev)}")
    print(f"Reversed - Validation set size: {len(y_val_rev)}")
```

```
    # apply nonlinear transformation
    Z_train_full_rev = transform(X_train_raw_rev)
    Z_val_full_rev = transform(X_val_raw_rev)

    validation_errors_rev = {}
    trained_models_rev = {}

    for k in k_values:
        # use only first k+1 features
        Z_train_rev = Z_train_full_rev[:, :(k+1)]
        Z_val_rev = Z_val_full_rev[:, :(k+1)]

        # train regression on training set
        w = solve_linear_regression(Z_train_rev, y_train_rev)

        # validation error
        e_val = calculate_classification_error(Z_val_rev, y_val_rev,
w)
        validation_errors_rev[k] = e_val
        trained_models_rev[k] = w

        print(f"k = {k}: E_val = {e_val:.6f} ({int(e_val *
len(y_val_rev))}/{len(y_val_rev)} errors)")

    best_k_validation_rev = min(validation_errors_rev,
key=validation_errors_rev.get)
    print(f"\nBest model (smallest val error): k =
{best_k_validation_rev}")
    print(f"Validation error:
{validation_errors_rev[best_k_validation_rev]:.6f}")

    # P4
    print("\nProblem 4")

    test_errors_rev = {}

    for k in k_values:
        # use only first k+1 features for test set
        Z_test = Z_test_full[:, :(k+1)]

        # use model trained in P3
        w = trained_models_rev[k]

        # calculate eout
        e_out = calculate_classification_error(Z_test, y_test, w)
        test_errors_rev[k] = e_out

        print(f"k = {k}: E_out = {e_out:.6f} ({int(e_out *
len(y_test))}/{len(y_test)} errors)")
```

```
    # find best model
    best_k_test_rev = min(test_errors_rev, key=test_errors_rev.get)
    print(f"\nBest model (smallest E_out): k = {best_k_test_rev}")
    print(f"E_out: {test_errors_rev[best_k_test_rev]:.6f}")

    # P5
    print("\nProblem 5")

    # get E_out for model chosen in P1
    e_out_p1 = test_errors[best_k_validation]
    print(f"E_out for model chosen in P1 (k={best_k_validation}):
{e_out_p1:.6f}")

    # get E_out for model chosen in P3
    e_out_p3 = test_errors_rev[best_k_validation_rev]
    print(f"E_out for model chosen in P3 (k={best_k_validation_rev}):
{e_out_p3:.6f}")

    # euclidean distance
    distance = np.sqrt((e_out_p1 - 0)**2 + (e_out_p3 - 0)**2)
    print(f"\nEuclidean distance: sqrt({e_out_p1:.6f}^2 +
{e_out_p3:.6f}^2) = {distance:.6f}")
    print(f"Individual values: ({e_out_p1:.1f}, {e_out_p3:.1f})")
```

```
Training set size: 25
Validation set size: 10
Test set size: 250

Problem 1
k = 3: E_val = 0.300000 (3/10 errors)
k = 4: E_val = 0.500000 (5/10 errors)
k = 5: E_val = 0.200000 (2/10 errors)
k = 6: E_val = 0.000000 (0/10 errors)
k = 7: E_val = 0.100000 (1/10 errors)

Best model (smallest val error): k = 6
Validation error: 0.000000

Problem 2
k = 3: E_out = 0.420000 (105/250 errors)
k = 4: E_out = 0.416000 (104/250 errors)
k = 5: E_out = 0.188000 (47/250 errors)
k = 6: E_out = 0.084000 (21/250 errors)
k = 7: E_out = 0.072000 (18/250 errors)

Best model (smallest E_out): k = 7
Out-of-sample error: 0.072000

Problem 3
Reversed - Training set size: 10
```

```
Reversed - Validation set size: 25
k = 3: E_val = 0.280000 (7/25 errors)
k = 4: E_val = 0.360000 (9/25 errors)
k = 5: E_val = 0.200000 (5/25 errors)
k = 6: E_val = 0.080000 (2/25 errors)
k = 7: E_val = 0.120000 (3/25 errors)

Best model (smallest val error): k = 6
Validation error: 0.080000

Problem 4
k = 3: E_out = 0.396000 (99/250 errors)
k = 4: E_out = 0.388000 (97/250 errors)
k = 5: E_out = 0.284000 (71/250 errors)
k = 6: E_out = 0.192000 (48/250 errors)
k = 7: E_out = 0.196000 (49/250 errors)

Best model (smallest E_out): k = 6
E_out: 0.192000

Problem 5
E_out for model chosen in P1 (k=6): 0.084000
E_out for model chosen in P3 (k=6): 0.192000

Euclidean distance: sqrt(0.084000^2 + 0.192000^2) = 0.209571
Individual values: (0.1, 0.2)
```

Problems 8-10

```python
from sklearn import svm

def generate_target_function():
    """
    random tangent in [-1,1] x [-1,1]
    """
    p1 = np.random.uniform(-1, 1, 2)
    p2 = np.random.uniform(-1, 1, 2)
    return p1, p2

def classify_point(x, p1, p2):
    """
    classify x according to tangent line
    """
    cp = (p2[0] - p1[0]) * (x[1] - p1[1]) - (p2[1] - p1[1]) * (x[0] -
p1[0])
    return 1 if cp > 0 else -1

def generate_dataset(N, p1, p2):
```

```python
    """
    generate and label N random points per target line
    """
    X = np.random.uniform(-1, 1, (N, 2))
    y = np.array([classify_point(x, p1, p2) for x in X])
    return X, y

def run_pla(X, y, max_iterations=10000):
    N, d = X.shape
    # bias term
    X_aug = np.c_[np.ones(N), X]
    w = np.zeros(d + 1)

    iteration = 0
    while iteration < max_iterations:
        predictions = np.sign(X_aug @ w)
        predictions[predictions == 0] = -1
        misclassified = np.where(predictions != y)[0]
        if len(misclassified) == 0:
            break
        idx = np.random.choice(misclassified)

        w += y[idx] * X_aug[idx]
        iteration += 1

    return w

def run_svm(X, y):
    """
    run SVM; hard margin
    """
    # use large C for hard margin
    clf = svm.SVC(kernel='linear', C=1e10)
    clf.fit(X, y)

    # weights and bias
    w = clf.coef_[0]
    b = clf.intercept_[0]

    # number of sv's
    n_support = len(clf.support_vectors_)

    return w, b, n_support

def measure_disagreement_pla(w_pla, p1, p2, n_samples=10000):
    """
    get diff between PLA and target function
    """
    X_test = np.random.uniform(-1, 1, (n_samples, 2))
```

```python
    # classify according to target function
    y_target = np.array([classify_point(x, p1, p2) for x in X_test])

    # classify according to PLA hypothesis (w includes bias)
    X_test_aug = np.c_[np.ones(n_samples), X_test]
    y_pred = np.sign(X_test_aug @ w_pla)
    y_pred[y_pred == 0] = -1

    disagreement = np.mean(y_pred != y_target)
    return disagreement

def measure_disagreement_svm(w_svm, b_svm, p1, p2, n_samples=10000):
    """
    get diff between SVM and target function
    """
    X_test = np.random.uniform(-1, 1, (n_samples, 2))

    # classify according to target function
    y_target = np.array([classify_point(x, p1, p2) for x in X_test])

    # classify according to SVM hypothesis
    y_pred = np.sign(X_test @ w_svm + b_svm)
    y_pred[y_pred == 0] = -1

    disagreement = np.mean(y_pred != y_target)
    return disagreement

def run_experiment(N, n_runs=1000):
    svm_better_count = 0
    support_vectors_counts = []
    valid_runs = 0

    while valid_runs < n_runs:
        # generate target function
        p1, p2 = generate_target_function()

        # generate dataset
        X, y = generate_dataset(N, p1, p2)

        # check if all points are on one side
        if len(np.unique(y)) == 1:
            continue

        valid_runs += 1

        # PLA
        w_pla = run_pla(X, y)
        disagreement_pla = measure_disagreement_pla(w_pla, p1, p2)

        # SVM
```

```
        w_svm, b_svm, n_support = run_svm(X, y)
        disagreement_svm = measure_disagreement_svm(w_svm, b_svm, p1,
p2)

        # compare
        if disagreement_svm < disagreement_pla:
            svm_better_count += 1

        support_vectors_counts.append(n_support)

        if valid_runs % 100 == 0:
            print(f"{valid_runs} runs completed")

    percentage_svm_better = 100 * svm_better_count / n_runs
    avg_support_vectors = np.mean(support_vectors_counts)

    return percentage_svm_better, avg_support_vectors


if __name__ == "__main__":
    # P8
    print("Problem 8")
    percentage_8, _ = run_experiment(10, 1000)
    print(f"SVM better than PLA: {percentage_8:.1f}%")
    print()

    # P9 and P10
    print("Problems 9 and 10")
    percentage_9, avg_sv_10 = run_experiment(100, 1000)
    print(f"SVM better than PLA: {percentage_9:.1f}%")
    print(f"Average support vectors: {avg_sv_10:.1f}")
```

```
Problem 8
100 runs completed
200 runs completed
300 runs completed
400 runs completed
500 runs completed
600 runs completed
700 runs completed
800 runs completed
900 runs completed
1000 runs completed
SVM better than PLA: 62.7%

Problems 9 and 10
100 runs completed
200 runs completed
300 runs completed
400 runs completed
```

```
500 runs completed
600 runs completed
700 runs completed
800 runs completed
900 runs completed
1000 runs completed
SVM better than PLA: 65.5%
Average support vectors: 3.0
```